Тема: Исследование и разработка специализированного исполнительного контура (Harness) для генеративного проектирования и верификации распределенных программных систем (Generative System Design)

1. Описание домена

Проблема базовых LLM

Когда разработчик или архитектор просит современную языковую модель («голую» LLM) спроектировать распределенный сервис (например, аналог такси, стриминга или маркетплейса), модель генерирует правдоподобный текст из терминов: рекомендует взять Kafka, Redis и шардировать PostgreSQL.

Однако этот текст зачастую лишь имитация:

- **Модель не умеет считать нагрузки:** она ошибается на порядки в оценке пропускной способности сети, дисковых операций (IOPS) и объема оперативной памяти.
- **Модель игнорирует фундаментальные законы распределенных систем:** она не видит противоречий (например, обещает строгую консистентность данных при асинхронной репликации в несколько датацентров).
- **Модель не видит каскадных сбоев:** она не может предсказать, что при падении кэша база данных мгновенно откажет из-за лавины запросов (*cache stampede*).

Что я предполагаю можно сделать

Создать для языковой модели **исполнительный контур (Harness)** - внешнюю инженерную среду, которая превращает LLM из «генератора текста» в **автоматизированный САПР (CAD) для программных архитекторов**.

В таком контуре архитектура рассматривается не как текст, а как **формальный исполняемый граф компонентов** (Architecture as Code):

- **Диспетчер (Root Agent)** разбирает задачу и формулирует жесткие требования к системе.
- **Субагенты** решают узкие задачи: один генерирует топологию сети, второй рассчитывает емкости, третий ищет единые точки отказа (SPOF).
- **Динамические навыки (Skills)** передаются агентам проверенные инженерные правила (формулы сайзинга, паттерны отказоустойчивости).
- **Песочница-симулятор (Sandbox)** берет созданный граф и запускает его в кодовом симуляторе, прогоняя виртуальный поток запросов и моделируя аварии. Ошибки симулятора возвращаются агентам для исправления топологии в замкнутом цикле рефлексии.

2. Первые предположения о бенчмарке (SystemDesign-Bench)

Главная сложность отсутствие однозначного «правильного» ответа. Чтобы исследование имело ценность, бенчмарк должен оценивать систему не субъективным мнением человека, а **математически верифицируемыми критериями**.

Набор эталонных сценариев (Dataset)

Планируется собрать и формализовать **20-30 эталонных задач**, разбитых на 3 уровня сложности:

1. **Базовый уровень (Single-service / Micro-pattern):** распределенный Rate Limiter, сокращатель ссылок с защитой от коллизий, кэширующий слой для тяжелой БД.
2. **Средний уровень (Multi-tier systems):** сервис ленты новостей на 50 000 RPS, система сквозного биллинга транзакций, аналитика кликов в реальном времени.
3. **Продвинутый уровень (Global-scale & High-resilience):** глобальный сервис вызова такси с гео-шардированием, распределенный стриминг видео с переменным битрейтом под SLA 99.99%.

Для каждой задачи задаются строгие входные параметры (SLA/SLO): целевой RPS на чтение/запись, лимит бюджета на инфраструктуру ($/мес), допустимая сетевая задержка (p99 latency <50ms) и модель согласованности данных.

Метрики оценки качества (Ground Truth)

- **Constraint Satisfaction Rate (CSR, %):** процент нефункциональных требований из ТЗ, которые реально соблюдены в архитектуре (рассчитанные параметры дисков, CPU, RAM и сети соответствуют формулам физических ограничений оборудования).
- **Graph Validity & Structural Soundness (%):** отсутствие висячих узлов, циклических синхронных блокировок и несовместимых стыков протоколов в архитектурном графе.
- **Simulation Pass Rate & Failure Resilience (FRS):** выдерживает ли архитектура проверку в симуляторе при пиковой нагрузке и при отключении одного произвольного узла (Chaos-тестирование).
- **Накладные расходы (Token & Latency Overhead):** замер стоимости и времени работы Harness относительно одного прямого вызова LLM для доказательства баланса между качеством и ценой вычислительных ресурсов.

3. Итоговый результат исследования к защите ВКР

К весне 2027 года формируется законченный программно-исследовательский комплекс:

1. **Модульный открытый фреймворк Harness (Python):**

- Реализация иерархии субагентов (генератор топологии, планировщик емкостей, аудитор надежности);
- Модульный каталог предметных навыков (/skills) с механизмами их runtime-инъекции;
- Алгоритмы рефлексии (Self-Correction Loop) на основе обратной связи от симулятора.

2. **Исполняемая песочница (Simulation Sandbox):**
   - Аналитический валидатор емкостей и сетевых потоков на Python;
   - Дискретно-событийный симулятор очередей (на базе SimPy), способный моделировать задержки, переполнение буферов и каскадные отказы предложенной топологии.
3. **Открытый бенчмарк (SystemDesign-Bench):**
   - Репозиторий задач, формализованных спецификаций и автотестов для воспроизводимой проверки любых языковых моделей в роли архитекторов.
4. **Демонстрационный стенд (Web/CLI):**
   - Визуальный интерфейс, наглядно демонстрирующий работу Harness в реальном времени: получение ТЗ → работа субагентов → запуск симуляции → нахождение узкого места → автоматическая перестройка архитектуры в отказоустойчивый вид.

4. Примерные этапы реализации на учебный год

Осенний семестр 2026: Фундамент, Бенчмарк и Аналитический Harness

- **Сентябрь - Октябрь: Формализация и проектирование DSL**
  - Разработка схемы формального графа архитектуры (на базе Pydantic-схем: узлы, типы очередей, базы данных, сетевые каналы).
  - Формирование первой версии бенчмарка (20 сценариев с эталонными SLA).
- **Ноябрь: Снятие Baseline и разработка ядра Harness**
  - Замер качества базовых LLM без обвязки на бенчмарке (снятие стартовых метрик).
  - Разработка ядра диспетчера и протокола динамической загрузки навыков (/skills).
  - Реализация связки специализированных субагентов (Topology Designer + Auditor).
- **Декабрь - Январь: Аналитическая песочница и промежуточный отчет**

    - Внедрение аналитического валидатора (математический расчет формул емкостей в Python-песочнице).
    - Замыкание первого контура рефлексии: автоматическое исправление расчетов при выявлении дефицита ресурсов.
    - Фиксация промежуточных результатов и подготовка отчета за 1-й семестр.

Весенний семестр 2027: Динамическая симуляция, Chaos Engineering

- **Февраль - Март: Разработка симуляционной песочницы**
    - Создание движка дискретно-событийного моделирования (SimPy/NetworkX) для прогона графа под нагрузкой.
    - Реализация сценариев Chaos Engineering: автоматическая симуляция отказа ноды, деградации сети или холодного старта кэша.
    - Интеграция телеметрии симулятора обратно в промпты агентов-исправителей.
- **Апрель: Эксперименты и абляционные исследования**
    - Масштабный прогон бенчмарка: Harness против базовой LLM, замер метрик CSR и Resilience.
    - Проведение абляционных тестов: оценка вклада каждого отдельного компонента (как влияют навыки, изоляция субагентов, симулятор).
- **Май - Июнь: Стенд**
    - Сборка финального демонстрационного стенда с визуализацией графов.

5. Анализ существующих решений на рынке (состояние на 2026 год)

Группа 1. Генераторы визуальных диаграмм (Eraser AI / DiagramGPT, ArchitectureDiagram.ai, IcePanel)

- **Что делают:** переводят текстовый запрос пользователя в схемы (Mermaid, PlantUML, C4 model, AWS/GCP stencils) или генерируют документацию по готовому коду.
- **Ограничения:** это исключительно **инструменты визуализации («косметика»)**. Они рисуют стрелочки между базами данных и сервисами, но не обладают семантическим пониманием распределенных систем: не проверяют, справится ли выбранная шина данных с пиковым RPS, сколько памяти нужно под кэш и не возникнет ли каскадный сбой при рестарте пода.

Группа 2. Мультиагентные среды разработки (MetaGPT, ChatDev, Devin, OpenHands)

- **Что делают:** содержат встроенную роль «Software Architect» в составе виртуальной команды разработки.
- **Ограничения:** роль архитектора сводится к **генерации текстовых Markdown-файлов** (структура папок, пользовательские истории, высокоуровневый PRD). Эти фреймворки сфокусированы на написании кода (SWE-уровень), полностью игнорируя нефункциональные требования высокой нагрузки (NFR: IOPS дисков, задержки репликации, сетевой оверхед, SLA).

Группа 3. Научные бенчмарки и архитектурные фреймворки (ArchBench, EngDesign, C4InterFlow)

- **Что делают:** C4InterFlow развивает концепцию Architecture as Code (AaC). Исследовательские бенчмарки (например, ArchBench 2026 года) оценивают качество генерации архитектурных записей решений (ADR) и бессерверных функций.
- **Ограничения:** ArchBench оценивает качество архитектуры через текстовые метрики схожести (BLEU, ROUGE, BERTScore) или запуск тривиальных unit-тестов, а не через поведение распределенной системы под нагрузкой. Нет среды, проверяющей динамику очередей и отказоустойчивость всей топологии.